TRANSFORMED 2 SUCCEED

transformed2succeed.com

FREE LIVE SESSION · 30 MINUTES

AI Agent Operations

Make AI agents reliable, secure and cost-effective in production. Everyone teaches build. We teach operate.

PRESENTER Dr. Emmanuel Naweji **LIVE** Mondays & Thursdays · 7:30 PM Central · 30 min

TONIGHT'S 30 MINUTES

Welcome! Here's the plan

0–2 min

Welcome

Say hi in the chat: your name, where you're joining from, and your role.

2–20 min

Three modules

One real idea from each module of AI Agent Operations, with the lab behind it.

20–23 min

Keep going

How to get the full course, every lab, and support along the way.

23–30 min

Your questions

Live Q&A. Drop questions in the chat any time; I'll answer at the end.

WHY THIS TRACK MATTERS

Most teams can build an AI agent. Few can run one.

01

Costs drift

Agent spend climbs without warning. Nothing crashes, so nothing alerts.

02

Behaviour changes

Agents can give a different answer each run. Traditional monitoring can't see it.

03

New risks appear

Tool-using agents bring threats standard security controls were never built for.

WHY NOW

Demand is growing far faster than the skills

151

US job postings naming “agentic systems” in 2024

16,500+

the same postings in 2025 (Stanford AI Index)

$185K+

typical base pay for these roles: $185K to $320K

The real bottleneck is no longer building models. It is reliability: making AI behave predictably, safely and affordably.

THE TRACK MAP

Three courses, in the recommended order

MODULE 1

Your First Production Agent

Build a working agent, then see the five things a happy-path demo never tests.

MODULE 2

The Agent Cost Problem

Measure, forecast and bound what your agent really costs, before it reaches the CFO.

MODULE 3

Running AI Agents in Production

The flagship: identity, budget caps, checkpointed runs and day-to-day operations.

01

MODULE 1 OF 3

Your First Production Agent

A tutorial agent works on the happy path. Production is everything else.

Not a chatbot. Not a workflow. A loop.

CHATBOT · WORKFLOW

You decide the path

prompt → answer, or step 1 → step 2 → step 3. The path was written in advance.

VS

AN AGENT

observe → think → act → repeat

The model decides the next step at run time. You write the tools and the loop, and when it stops.

Who decides when it stops? The model says it's done. A max number of steps: often missing. Something goes wrong: rarely handled.

The model never runs anything

1

Your code sends the chat

System prompt, tool schemas, and every message so far.

2

The model replies tool_use

A tool name and arguments. Nothing has happened yet.

3

Your code runs the tool

With your permissions, then sends back a tool_result.

4

The model decides again

Another tool_use, or end_turn: done.

Run the ticket-triage agent for free

TERMINAL

```
$ git clone https://github.com/Here2ServeU/first-production-agent
$ cd first-production-agent
$ cd modules/module-1/scripts
$ python3 run_agent.py --input samples/normal.json --sim
```

`--sim`
simulated model: free, local, repeatable

`--trace`
prints tool_use, tool_result and stop_reason

`samples/refund.json`
harder ticket: 6 steps, no code change

YOU SHOULD SEE lookup_customer → search_kb → route_ticket, then DECISION: reply (article #214) · steps: 3 · stop_reason: end_turn

Five questions the demo cannot answer

01 **What did that cost?**
And what stops it if a ticket never finishes?

02 **Who ran it?**
Six months later, can you prove what it did?

03 **Was the answer right?**
Exit code 0 is not the same as correct.

04 **What if input is hostile?**
The ticket text goes straight to the model.

05 **What if it dies at step 4?**
Start again, pay again, act twice?

Build it. Break it. Name it. Each failure is shown on your machine and pointed at the course that fixes it.

Your mission: build one, break five, diagnose

1

Build: module 1

Run the working agent, read its trace, count its steps.

2

Break: modules 2 to 6

One failure each: loop, audit, correctness, hostile input, crash.

3

Measure the damage

Real numbers from your own machine, not mine.

4

Diagnose: capstone

A production-readiness report, with evidence for every line.



Working is not the same as ready for production.

Today's honest score: 0 / 5. Zero is the right answer. Would you let this run while you sleep?

02

MODULE 2 OF 3

The Agent Cost Problem

Traditional compute cost scales with traffic. Agent cost scales with reasoning.

One malformed record. One long night.

23:40

a malformed record enters the queue

~400

retries an hour

9.5 hrs

before anyone noticed

$4,200

of overnight spend, with 0 alerts

Every dashboard stayed green. The agent worked exactly as designed. Cost was the only signal that moved, and nothing watched it.

Traditional cost adds. Agent cost multiplies.

TRADITIONAL

requests × cost per request

Linear. Napkin math. More traffic, more cost. Every monitoring tool assumes this.

VS

AGENT

requests × steps × tokens × retries

You control requests. The work decides steps, tokens and retries. A retry repeats the whole tree.

Input tokens grow every step: step 12 resends everything from steps 1 to 11. Long runs cost more than you think.

Same afternoon. Same money. Two stories.

100

requests, $19 spent in total

62

succeeded; 38 failed after consuming tokens

4¢

cost per request: “sounds fine”

19¢

cost per successful task: nearly 5× higher, and true

Cost per successful task puts failures in the denominator. It's the only unit that survives a conversation with finance.

Price one healthy run

TERMINAL

```
$ git clone https://github.com/Here2ServeU/agent-cost-control
$ cd agent-cost-control/modules/module-1/scripts
$ python3 run_agent.py --input samples/normal.json --verbose
$ ... | python3 price_run.py --input-rate 0.80 --output-rate 4.00
```

`--verbose`
prints each call's input_tokens and output_tokens

`price_run.py`
adds up the usage lines and prices them

`--input-rate`
your real model's prices, $ per million tokens

YOU SHOULD SEE 6 calls · 15,137 input = $0.0454 · 2,119 output = $0.0318 · run $0.0772 (default rates). Token ratio 7.1:1, cost ratio 1.4:1.

Your mission: extrapolate your own $4,200

1

Price a healthy run

Steps, tokens, cost. Three numbers, written down.

2

Run the one that can't finish

malformed.json with a hard 120-second limit. Never remove it.

3

Extrapolate

Calls in 2 min × cost per call × 30 = one hour, then × 9.5 hours.

4

Ask the budget owner

“Is that an acceptable worst case?”



Reasoning is unbounded; you have to bound it.

Bounded · Visible · Attributable. Report cost per successful task.

03

MODULE 3 OF 3

Running AI Agents in Production

The flagship: deployment, reliability, failure handling and day-to-day operations.

What a production agent needs that a demo doesn't

01
Per-agent identity
Each agent runs as itself, not one shared API key, so you can prove who ran what.

02
Budget caps
A spending limit the agent cannot cross, on every run.

03
Checkpointed runs
Save work as it goes, so a restart doesn't begin from zero.

04
Drain on rollout
Ship a new version without cutting runs off halfway through.

05
Behavioural baselines
Know what normal looks like, so you can spot when it changes.

06
Per-integration health
Health metrics for each tool and system the agent depends on.

From a demo agent to an operated agent

THE DEMO AGENT

Score: 0 / 5

Shared key, no audit trail, no ceiling. Dies at step 4 and starts over. Nobody knows what normal is.

VS

THE OPERATED AGENT

Every question has an answer

Own identity, capped budget, checkpoints, safe rollouts, baselines, and health per integration.

Fewer production incidents: proven reliability practices replace trial-and-error.

Start from your own scorecard

1

Who ran it?

Give every agent its own identity and an audit trail.

2

What if it dies at step 4?

Checkpoint runs, and drain them safely on every rollout.

3

What stops it?

Budget caps on every run, from course two.

4

Is it still behaving?

Baselines and per-integration health, watched daily.



Frameworks churn. Reliability engineering endures.

Teach the operations discipline, not a framework. It outlasts the hype, like cloud and data engineering did.

RECAP

Three courses, one operating discipline

Working is not production-ready

Course 1: an agent is a loop. The model asks; your code acts. Score: 0 / 5.

Reasoning is unbounded: bound it

Course 2: cost multiplies. Report cost per successful task.

Operate, don't just build

Course 3: identity, budget caps, checkpoints, safe rollouts, baselines.



“Would I let this run while I sleep?”

By the end of the track, you can name every reason it's not safe yet, measure it, and fix it.

THE MEMBERSHIP

Every course. One price.

$49

per month

All 8 courses, every module and every lab, plus a live session with me every month.

ALL 8 COURSES INCLUDED

- Linux for AI-ML Systems
- Python for AI-ML
- Zero to AI-ML Systems Engineering
- DevOps for AI-ML Systems
- SRE for AI-ML Systems
- GPU Optimization for AI-ML Systems
- FinOps for AI-ML Systems
- AI Agent Operations

BUILD WITH ME, LIVE

Once a month, we build something together

1

Learn at your pace

Work through any course, any module, on your own schedule.

2

Bring a real problem

An idea, a project at work, or a lab you want to take further.

3

Meet me live

Once a month we get on a call and build it together, step by step.

4

Leave with it built

Something real you made, understand and can show.

FREE LIVE SESSIONS

Every Monday and Thursday, 7:30 PM Central

A different course each session. The full rotation takes 4 weeks, then starts again.

	WEEK 1	WEEK 2	WEEK 3	WEEK 4
MONDAY	NEXT Linux for AI-ML Systems	Zero to AI-ML Systems Engineering	SRE for AI-ML Systems	FinOps for AI-ML Systems
THURSDAY	Python for AI-ML	DevOps for AI-ML Systems	GPU Optimization for AI-ML Systems	TONIGHT AI Agent Operations

Join the membership today

$49/month · all 8 courses · a live build session with me every month

transformed2succeed.com

Next free session: Monday, 7:30 PM Central · Linux for AI-ML Systems

Questions? Let's take them now.

Dr. Emmanuel Naweji · Transformed 2 Succeed